Autonomous Smart Waste Collection Robot with Sensor-Based Bin Fullness Detection and Dynamic Route Optimization

Cansu Aslan
Department of Computer Engineering
Hacettepe University
Ankara, Türkiye
2210356079

Gökdeniz Şimşek
Department of Computer Engineering
Hacettepe University
Ankara, Türkiye
2210356067

Mert Can Köseoğlu
Department of Computer Engineering
Hacettepe University
Ankara, Türkiye
2220356055

Zeynep Nisa Karataş
Department of Computer Engineering
Hacettepe University
Ankara, Türkiye
2210356066

Prof. Dr. Mehmet Önder EFE
Department of Computer Engineering
Hacettepe University
Ankara, Türkiye
Supervisor

Abstract*—Waste collection in campus and indoor-like environments is commonly performed through fixed routes, regardless of whether waste bins are full or empty. This approach causes unnecessary travel, increased energy consumption, and inefficient use of human labor. In this study, an autonomous smart waste collection robot is developed to detect the fullness status of waste bins and dynamically plan an optimized route only toward the bins that require service. The proposed system is implemented on the Mobo Robo mobile robot platform using ROS Noetic. A two-dimensional map of the environment is created using LiDAR data and SLAM Toolbox. Waste bin locations are defined as navigation nodes on the generated map, and a graph-based representation is constructed. A cost matrix is generated between the nodes, and a Dijkstra-like shortest path algorithm is used to determine the optimal visit order for the bins that report fullness alarms. For bin monitoring, ultrasonic and load cell sensors are used together. The ultrasonic sensor estimates the fill percentage based on distance measurements, while the load cell measures the bin weight. The sensor unit transmits bin status data in JSON format through MQTT communication. The robot-side route executor subscribes to these messages and updates the active target list accordingly. The system also supports interruptible dynamic replanning, allowing the robot to update its route when a new alarm is received during navigation. Experimental results show that the proposed architecture successfully integrates sensor-based monitoring, MQTT-based communication, LiDAR-based mapping, ROS-based navigation, and graph-based route optimization into a working autonomous waste collection prototype.

***Index Terms*—Autonomous robot, smart waste collection, ROS Noetic, SLAM Toolbox, MQTT, ultrasonic sensor, load cell, Dijkstra algorithm, route optimization.**

I. Introduction

Waste management is an important operational challenge in campuses, public buildings, industrial areas, and smart city environments. As the number of users in such environments increases, the amount of daily waste also increases. Traditional waste collection systems usually depend on fixed schedules and predefined routes. In these systems, cleaning personnel or collection vehicles visit all waste bins in a specific area, regardless of whether they are full, partially full, or empty. Although this method is simple to implement, it causes inefficiency in terms of time, energy, labor, and operational cost.

The main problem addressed in this project is the inefficiency of fixed-route waste collection. In a fixed-route system, the collection unit spends time checking bins that may not require service. This causes unnecessary movement and increases the total route length. Furthermore, fixed-route collection systems cannot react to sudden changes, such as a bin becoming full earlier than expected. Therefore, there is a need for an intelligent waste collection system that can detect the current status of bins and dynamically plan its route according to real-time information.

Recent studies in smart waste management have focused on sensor-based monitoring, Internet of Things (IoT) communication, and route optimization. Hannan et al. proposed a capacitated vehicle routing model for solid waste collection and showed that optimization-based approaches can reduce travel distance and improve collection efficiency [1]. Priyadarshi et al. studied dynamic routing for efficient waste collection in resource-constrained scenarios and emphasized that dynamic decision-making is important when waste generation is not uniform [2]. Martikkala et al. investigated an IoT-based smart textile waste collection system and demonstrated the value of combining sensor data with dynamic route optimization [3]. These studies show the importance of dynamic routing; however, many approaches remain at the level of vehicle routing, fleet management, or passive IoT monitoring rather than an integrated autonomous mobile robot system.

Robotic navigation has also been widely studied through

ROS-based frameworks. ROS provides a modular software architecture that allows different components such as mapping, localization, planning, and control to communicate through topics, services, and action interfaces [4]. SLAM Toolbox can be used for simultaneous localization and mapping, especially when a robot needs to construct or use a two-dimensional occupancy grid map [5]. The ROS navigation stack, including the `move_base` package, enables a mobile robot to receive goal positions and move toward them while considering global and local costmaps [6]. These tools provide a suitable foundation for autonomous waste collection.

In sensor-based bin monitoring, ultrasonic sensors are frequently used because they can measure the distance between the top of the bin and the waste surface. If the measured distance becomes smaller than a predefined threshold, the bin can be classified as full. However, relying only on an ultrasonic sensor may create false positives, especially when irregularly shaped waste is close to the sensor. Similarly, a load cell can measure bin weight, but weight alone may also be misleading because lightweight waste may occupy a large volume while heavy waste may occupy a small volume. For this reason, this project combines ultrasonic distance data and load cell weight data to make a more reliable fullness decision.

The research question of this study is: Can an autonomous mobile robot improve waste collection efficiency by using sensor-based bin fullness detection and dynamically recalculating its route according to real-time alarm messages? To answer this question, an integrated autonomous smart waste collection system is developed on the Mobo Robo platform using ROS Noetic. The system creates a map of the environment using LiDAR data and SLAM Toolbox, defines waste bin locations as graph nodes, constructs a cost matrix between these nodes, and applies a Dijkstra-like algorithm for route optimization. When a bin becomes full, the sensor unit sends a JSON-formatted MQTT alarm message to the robot computer. The robot updates its target list and calculates the most efficient path. If a new alarm is received during navigation, the system can interrupt the current route and recalculate the optimal path.

II. System Architecture

The proposed system consists of four main layers: mapping and navigation, bin status detection, communication, and route optimization. The mapping and navigation layer is implemented using ROS Noetic on the Mobo Robo mobile robot platform. The bin status detection layer uses an ultrasonic sensor and a load cell to determine whether a waste bin is full. The communication layer transfers the sensor decision to the robot computer using MQTT in JSON format. The route optimization layer uses a cost matrix and a Dijkstra-like shortest path algorithm to generate the optimal route between active waste collection nodes.

Fig. 1 shows the general structure of the proposed system. The sensor unit publishes bin status messages to the MQTT broker. The robot computer subscribes to the related MQTT topic, receives the alarm information, updates the active target list, computes the route using the cost matrix, and sends navigation goals to the ROS navigation stack.

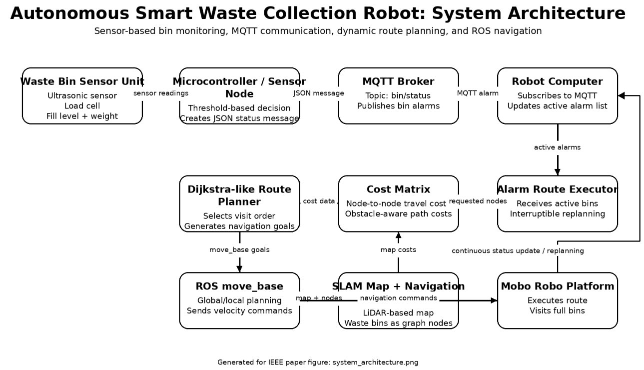

Fig. 1. General system architecture of the autonomous smart waste collection robot.

The general workflow of the system is as follows. First, the robot maps the environment using LiDAR data. Then, important locations such as the robot start point and waste bin positions are marked as nodes. After that, navigation costs between nodes are calculated and stored in a cost matrix. When a bin sends a fullness alarm, the corresponding node is added to the active target list. The route planner calculates the most efficient route from the robot's current location to the active bins. Finally, the robot sends navigation goals to the ROS navigation stack and follows the selected path.

III. Methodology

A. Mapping with LiDAR and SLAM Toolbox

The environment map is created using LiDAR data and SLAM Toolbox. LiDAR measurements provide distance information between the robot and surrounding obstacles. These measurements are used to construct a two-dimensional occupancy grid map. In this map, free spaces, occupied regions, and unknown areas are represented in a grid structure. The occupancy grid is then used as the basis for navigation and route planning.

After the map is generated, the coordinates of the waste bins are manually identified on the map. Each waste bin location is assigned as a navigation node. These nodes form the graph structure used in the route optimization stage. The robot's initial position is also included as a node so that the route can be calculated from the robot's current location to the active waste collection targets.

B. ROS Noetic Navigation Architecture

The robot navigation system is implemented using ROS Noetic. The `move_base` package is used to send navigation goals and control the robot's movement toward selected target points. The navigation system uses the map, robot localization data, and obstacle information to plan safe movement.

The global planner is responsible for producing a path from the robot's current position to the target node. The local planner is responsible for generating velocity commands while avoiding obstacles and respecting the robot's motion constraints. The robot moves between predefined waste bin nodes according to the order selected by the route optimization algorithm.

The modular structure of ROS allows different software components to operate as separate nodes. The main nodes in the system are the sensor data processing node, MQTT listener node, route planner node, and navigation command node. These nodes communicate through ROS topics, MQTT messages, and internal route planning logic.

C. Bin Fullness Detection

The bin fullness detection system uses two sensors: an ultrasonic sensor and a load cell. The ultrasonic sensor measures the distance from the top of the bin to the nearest surface of the waste inside the bin. If the measured distance is below a predefined threshold, the bin is considered to have a high fill level. The fill percentage is estimated as follows:

$$F = \frac{D_{empty} - D_{measured}}{D_{empty} - D_{full}} \times 100 \tag{1}$$

where F is the fill level percentage, D_{empty} is the distance measured when the bin is empty, $D_{measured}$ is the current ultrasonic distance, and D_{full} is the distance when the bin is considered full. A smaller measured distance indicates a higher fill level.

The load cell measures the weight of the bin. If the weight exceeds a predefined threshold, the bin is considered heavy enough to require collection. However, weight information alone is not sufficient because different types of waste may have different densities. Therefore, ultrasonic and load cell measurements are evaluated together.

The final fullness decision is made using a threshold-based fusion approach. A bin is classified as full when the fill percentage exceeds the selected fullness threshold and/or the weight exceeds the selected weight threshold. This combined approach aims to reduce false alarms compared with using only one sensor.

D. MQTT-Based Communication

The sensor unit transmits bin status information to the robot computer using MQTT communication. MQTT is selected because it is lightweight and suitable for real-time IoT-style communication. Each bin status message is sent in JSON format. A typical message contains the node ID, location information, measured weight, estimated fill percentage, and final fullness decision.

An example JSON message is shown below:


```
{
  "node_id": 1,
  "lat": 39.872101,
  "lng": 32.735199,
  "weightKg": 0.04,
  "fillPercent": 14,
  "isFull": false
}
```


The robot computer subscribes to the MQTT topic `bin/status`. When an alarm message is received, the corresponding bin node is added to the active target list. If the fullness status is false, the node is not included in the route.

E. Cost Matrix Construction

After the map and node positions are defined, a cost matrix is generated between the nodes. Each element of the matrix represents the travel cost between two nodes. The cost can be based on Euclidean distance, map-based path length, or obstacle-aware navigation cost. In this project, the cost matrix is used as the input of the route optimization algorithm.

If there are n nodes in the environment, the cost matrix C can be represented as:

$$C = \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1n} \\ c_{21} & c_{22} & \cdots & c_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n1} & c_{n2} & \cdots & c_{nn} \end{bmatrix} \tag{2}$$

where c_{ij} represents the travel cost from node i to node j. If two nodes are directly reachable, the corresponding cost is assigned according to the estimated travel distance or the path returned by the navigation planner. If a connection is not valid or safe, the cost can be assigned as infinity or a very large value.

F. Route Optimization Algorithm

A Dijkstra-like shortest path algorithm is implemented to calculate the optimal path between the robot and active waste collection nodes. Dijkstra's algorithm is suitable for graphs with non-negative edge weights [7]. Since travel costs between nodes are non-negative, it can be used to determine the minimum-cost path from the robot's current node to a target node.

The algorithm receives the current robot node, active alarm nodes, and the cost matrix as input. It calculates the shortest path from the robot's current location to each active alarm node. Then, the most suitable next target is selected according to the lowest path cost. After reaching a target node, the robot updates its current node and recalculates the next best target if other active alarms remain.

TABLE I
MAIN INPUTS AND OUTPUTS OF THE ROUTE PLANNER

Component	Description
Input 1	Current robot node
Input 2	Active alarm nodes received through MQTT
Input 3	Precomputed node-to-node cost matrix
Output 1	Optimized visit order
Output 2	Sequential `move_base` navigation goals

G. *Interruptible Dynamic Replanning*

One of the important parts of the system is the interruptible route planning mechanism. In a static route system, the robot would complete the current route even if a new bin becomes full. In the proposed system, the robot continuously listens for MQTT alarm messages during navigation.

If a new alarm is received while the robot is moving from one node to another, the route planner checks the updated active target list and recalculates the optimal route from the robot's current or nearest node. If the newly received alarm creates a more efficient route, the system updates the target order and sends a new goal to the navigation module. This makes the system responsive to real-time changes.

IV. Experimental Results and Discussion

A. *Mapping and Node Definition Results*

The first stage of the project was the creation of the environment map using LiDAR data and SLAM Toolbox. The robot successfully generated a two-dimensional occupancy grid map of the test environment. After the map was obtained, waste bin locations were marked as predefined navigation nodes. These nodes became the main reference points for route planning.

The map-based node definition was important because the route optimization algorithm requires a graph representation of the environment. Instead of planning over every point in the map, the system uses meaningful locations as graph nodes. This reduces computational complexity and makes the route planning process easier to control.

B. *Sensor-Based Fullness Detection Results*

The MQTT-based bin monitoring system was tested by transmitting sensor measurements and fullness information from the smart waste bins to the robot navigation system. Sensor data were published as JSON messages through the `bin/status` topic and included the bin node ID, geographic coordinates, measured weight, fill percentage, and fullness status.

Table II shows an example sensor message received during the experiments. In this example, the measured weight is 0.04 kg and the estimated fill level is 14%, resulting in a fullness decision of `false`.

TABLE II
Example MQTT Sensor Message

Parameter	Example Value
Node ID	1
Latitude	39.872101
Longitude	32.735199
Weight	0.04 kg
Fill percentage	14%
Fullness status	False

The results demonstrate that the MQTT communication infrastructure successfully transmitted bin status information in real time. This information was then used by the route planning module to determine which bins required service and to generate an optimized collection route for the mobile robot.

C. *MQTT Communication Results*

The MQTT communication module successfully transferred bin alarm information from the sensor side to the robot computer. The use of JSON messages made the communication structure readable and easy to extend. Each message included the bin node identity and the fullness status. When the robot computer received an alarm, the corresponding node was added to the active target list.

The main advantage of this communication design is modularity. The robot navigation system does not need to directly read sensor pins. Instead, it only listens to MQTT messages. This allows the sensor unit and the robot computer to work independently. It also makes the system easier to expand with multiple bins because each bin can publish its own status message.

D. *Route Optimization Results*

The route planner used the active alarm nodes and the precomputed cost matrix to determine the order in which the robot should visit the full bins. During an example execution, alarms were received from bins 2, 3, 4, and 6. Based on the travel costs between nodes, the system generated the following route:

$$bin\_4 \rightarrow bin\_6 \rightarrow bin\_3 \rightarrow bin\_2 \tag{3}$$

The generated route was sent sequentially to the ROS navigation stack through the `move_base` action server. The console output confirmed that the robot successfully reached each target node and completed the alarm collection route without errors.

Fig. 2 shows the robot-side route execution output. The output indicates that the static map costs were loaded, MQTT alarm input was enabled, the node distance matrix was built, alarms were queued, and the final alarm route was completed successfully.

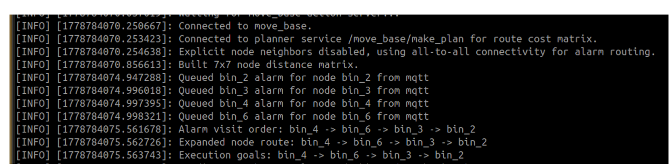

Fig. 2. Robot-side route executor output showing MQTT alarm reception and successful route completion.

The route optimization process reduced unnecessary movements by directing the robot only to bins that generated alarms. Unlike a fixed-route collection strategy, the robot did not need to visit bins that were not full. This behavior improves operational efficiency by reducing travel distance and navigation time.

E. Interruptible Replanning Results

The interruptible route planning mechanism was one of the most important functional results of the project. During navigation, the robot continued to listen for MQTT alarm messages. When a new alarm was received, the system updated the active target list and recalculated the route.

This feature made the robot more flexible than a static planner. For example, if the robot was moving toward a distant bin and a closer bin became full, the system could compare the new target with the current route and update the path if necessary. This behavior is especially important in real environments where bin statuses can change while the robot is already operating.

The main challenge in interruptible replanning is preventing unnecessary route changes. If the robot changes its target too frequently, it may become inefficient or unstable. Therefore, future versions may include a decision threshold. For example, the robot may only change its current target if the new route provides at least a certain percentage of improvement.

F. Overall System Demonstration

The integrated system demonstrated the complete workflow of autonomous smart waste collection. First, the robot used a previously generated map. Second, the bin sensor unit evaluated fullness status using ultrasonic and load cell data. Third, the sensor unit sent a JSON-formatted MQTT alarm. Fourth, the robot computer received the alarm and updated the active target list. Finally, the route planner selected the optimal node order and the robot navigated toward the required bin locations.

The system showed that ROS-based navigation, sensor-based bin monitoring, MQTT communication, and graph-based route planning can be combined in a single working prototype. The modular architecture also makes the system suitable for future improvements.

V. Limitations

Although the proposed system successfully demonstrates an integrated autonomous waste collection workflow, it has several limitations. First, the experiments were conducted in a controlled environment with a limited number of predefined nodes. In larger environments, the cost matrix may become more complex and route optimization may require more scalable algorithms. Second, the current fullness detection method is based on threshold-based sensor fusion. Although this approach is simple and practical, adaptive thresholds or machine learning-based classification could improve robustness under different waste types. Third, the system depends on wireless MQTT communication. If the Wi-Fi connection becomes unstable, alarm messages may be delayed or lost. Finally, real outdoor environments may introduce additional challenges such as uneven surfaces, dynamic obstacles, localization drift, and weather conditions.

VI. Conclusion and Future Work

In this study, an autonomous smart waste collection robot was developed using ROS Noetic, LiDAR-based mapping, sensor-based bin fullness detection, MQTT communication, and graph-based route optimization. The system was implemented on the Mobo Robo mobile robot platform. SLAM Toolbox was used to generate a two-dimensional map of the environment, and waste bin locations were represented as graph nodes. Ultrasonic and load cell sensors were used together to estimate bin fullness. The sensor unit transmitted JSON-formatted MQTT messages to the robot computer. The route planner used a precomputed cost matrix and a Dijkstra-like shortest path algorithm to determine the optimal visit order for active alarm nodes.

The experimental results showed that the proposed system successfully integrated sensing, communication, navigation, and route optimization into a working prototype. In an example execution, the robot received alarms from multiple bins and generated the route $bin\_4 \rightarrow bin\_6 \rightarrow bin\_3 \rightarrow bin\_2$. The robot then visited the selected nodes sequentially through the ROS `move_base` navigation stack and completed the alarm route successfully.

Future work can focus on testing the system in larger and more realistic environments, improving the fullness detection algorithm with adaptive thresholds or machine learning, adding battery-aware and priority-aware route planning, and developing multi-robot coordination. A cloud-based dashboard can also be added to monitor bin status, robot location, route history, and performance metrics in real time. Overall, the proposed system represents a step toward intelligent, autonomous, and sustainable waste collection for smart campuses and smart city applications.

Acknowledgment

The authors would like to thank Prof. Dr. Mehmet Önder EFE for supervising this project and providing guidance throughout the development process.

References

[1] M. A. Hannan, M. Akhtar, R. A. Begum, H. Basri, A. Hussain, and E. Scavino, “Capacitated vehicle-routing problem model for scheduled solid waste collection and route optimization using PSO algorithm,” *Waste Management*, vol. 71, pp. 31–41, 2018.

[2] M. Priyadarshi, A. Sharma, and R. Singh, “Dynamic routing for efficient waste collection in resource-constrained scenarios,” *Scientific Reports*, vol. 13, no. 1542, 2023.

[3] A. Martikkala, B. Mayanti, P. Helo, and A. Lobov, “Smart textile waste collection system–Dynamic route optimization with IoT,” *Journal of Environmental Management*, vol. 335, 2023.

[4] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, “ROS: an open-source Robot Operating System,” in *ICRA Workshop on Open Source Software*, vol. 3, no. 3.2, 2009, p. 5.

[5] S. Macenski and I. Jambrecic, “SLAM Toolbox: SLAM for the dynamic world,” *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.

[6] E. Marder-Eppstein, E. Berger, T. Foote, B. Gerkey, and K. Konolige, “The office marathon: Robust navigation in an indoor office environment,” in *2010 IEEE International Conference on Robotics and Automation*, 2010, pp. 300–307.

[7] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.